

Karima Zine

Lien Trello : <https://trello.com/b/pd5nRD5U/ecf-vite-gourmand>

Lien GitHub : <https://github.com/Rika600/Vite---Gourmand>

Gestion de projet Vite & Gourmand

Outil de gestion de projet

J'ai utilisé Trello pour organiser et suivre l'avancement du projet. Le tableau est organisé en 4 colonnes : "À faire" pour les tâches à venir, "En cours" pour ce sur quoi je travaille, "À tester" pour vérifier que tout fonctionne avant de valider, et "Terminé" pour les tâches validées. Cette organisation m'a permis de suivre l'avancement du projet de manière claire et de ne rien oublier.

Organisation du travail

J'ai organisé mon travail en plusieurs étapes. D'abord la compréhension : lire et analyser la consigne pour bien cerner les attentes du client. Ensuite, séparer les demandes en différentes étapes réalisables. Puis l'organisation du travail dans sa globalité : qu'est-ce qui est demandé ? Comment le faire ? Et où le faire ? Enfin, établir un plan de déroulement avec les étapes à suivre dans l'ordre.

Les étapes du projet ont été les suivantes :

Une fois l'énoncé compris, j'ai séparé les demandes avec leurs technologies respectives et vérifié l'environnement de travail afin d'avoir tous les outils nécessaires pour mener ce projet à bien.

Tout étant à jour, j'ai pu démarrer en organisant le déroulement du projet sur Trello. La première chose à faire fut le résumé du projet, puis le cahier des charges. Ensuite, la phase de conception : j'ai créé le diagramme UML (cas

d'utilisation), les diagrammes de séquence et le diagramme de classe sur Draw.io afin de définir les rôles et leurs possibilités. À ce stade, on a une vue d'ensemble : qui ? quoi ? et comment ?

Ensuite, les croquis du site sur papier, puis leur réalisation sur Figma avec la charte graphique (couleurs, polices, maquettes wireframes et mockups).

Une fois cela fait, il a fallu passer à la base de données. J'ai d'abord rédigé les tables sur VS Code pour garder une trace et pouvoir les modifier facilement, puis je les ai importées sur phpMyAdmin. Une fois les 15 tables créées et complètes, j'ai pu passer à la construction du site.

J'ai opté pour un vertical slicing, qui consiste à faire une page complète et opérationnelle avant de passer à la suivante. Avant de commencer les pages, j'ai créé un header et un footer réutilisables, inclus sur chaque page grâce à `require_once` en PHP. Cela évite de réécrire le même code sur chaque page et facilite les modifications. Chaque page a la même organisation SQL, PHP, HTML, CSS/SCSS avec Bootstrap 5, et JavaScript.

Puis j'ai installé Docker dans le projet afin de conteneuriser l'application. Cela permet de garantir que le site fonctionne de manière identique sur n'importe quel environnement, sans dépendre de la configuration de chaque machine. Docker a également facilité le déploiement en ligne sur Railway. De plus, l'isolation dans un conteneur limite l'exposition du système hôte, ce qui apporte une couche de sécurité supplémentaire.

Enfin, une fois le site et les tests terminés, j'ai utilisé l'hébergeur Railway pour le déploiement en ligne. Railway est gratuit, compatible avec PHP, MySQL et Docker, et sans limitation pour un projet de cette taille.

Suivi et versionnement

Tout au long du processus, l'organisation du travail a nécessité une documentation continue. Le code a été versionné sur Git et hébergé sur GitHub en dépôt public, avec trois branches : "develop" pour le développement, "feature/migration-php" pour les fonctionnalités, puis "main" pour la production finale. Les commits décrivent clairement les modifications apportées tout au long.